

A Rudimentary Intro to C programming

Wayne Goddard

School of Computing, Clemson University, 2025

Part 5: Odds and Ends

22	Files and I/O	E1
23	Command Line Arguments and Header Files	E3
24	BitOps	E6

CHAPTER 22

Files and I/O

22.1 Streams and Redirection

A *stream* is a source of input or a destination of output. One automatically has the standard streams `stdin` and `stdout`; we have accessed these with `scanf` and `printf` etc. One can redirect these. For example:

```
a.out < in.txt > out.txt
```

will cause the program to read from `in.txt` (instead of the keyboard) and write to `out.txt` (instead of console).

22.2 Files

In C, a `FILE` provides access to a stream. It is declared with

```
FILE *myfile;
```

You just need to know the basics. (But to be honest I have to look up these every time I use them.) The four common operations are:

- `fopen(fileName, accessString)` — *open a stream*:
This creates access to the file on disk with the specified name and returns a *handle* (so one writes `myfile = fopen(...)`). The access string specifies the *mode*: "r" for reading and "w" for writing. (Can also specify other modes.) Note that the handle is returned as `NULL` (0) if there was a failure; one should check this.
- `fgets(string, maxSize, myfile)` — *read from a stream*:
This function reads a line from the stream into the `string` and appends the `'\0'`-terminator. The number of characters read had better be at most `maxSize - 1`. The function returns `NULL` if end-of-file prevented reading.
- `fprintf(myfile, string, vals)` — *write to a stream*:
This is like `printf`, but to the stream.
- `fclose(myfile)` — *close a stream*:
This is called at the end. It helps the operating system by releasing resource.

The formal signatures are:

```
FILE *fopen( const char *name, const char *mode );
int fclose( FILE *f );
char *fgets( char *buffer, int buffer_size, FILE *stream);
int fprintf( FILE *stream, const char *format, ... );
```

There are also: `feof`, `fputs`, `fputc`, `fgetc`. Beware: do not use `open()` or `close()`; these are low-level commands.

22.3 Sample Code

For copying from one file to another.

```
// filer.c copies from one file to another
// not robust
#include <stdio.h>

int main(void) {
    FILE *in, *out;
    in = fopen("from.txt", "r");
    out = fopen("to.txt", "w");
    if( !in || !out ) {
        printf("Error in opening files\n");
        return 1;
    }
    char s[100];
    while( fgets(s, 100, in)!=NULL )
        fprintf(out,s);
    fclose( in );
    fclose( out );
    return 0;
}
```

Practice. Write a program that will take the input, and write it to a file adding line-numbers.

Command Line Arguments and Header Files

23.1 Command Line Arguments

A C program can access information you type on the *command line*. This is useful, for example, if you want to specify the name of an input file at run time.

For example we could do the following:

```
a.out myDict.txt
```

Using a different signature, the main function has access to the command line as an array of strings. The array is called `argv` and the count is given by `argc`. In the above case, it would get an array with two strings: “a.out” and “myDict.txt”.

Here is code, taken from howstuffworks.com, which echoes the command line to the screen. Note that `char *argv[]` is an array of strings.

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    int x;
    printf("argc is %d\n", argc);
    for (x=0; x<argc; x++)
        printf("arg[%d] is %s\n", x, argv[x]);
    return 0;
}
```

Practice. Write a program that will add up all the values in a file whose name is supplied by the user on the command line.

23.2 Header Files

You have included several files ending in “.h”. These are *header* files. They do not have the source code, only the prototypes. You can do the same. We illustrate this with an example.

Suppose one wanted to write a statistics library, and a program that uses it. The library would contain functions for computing and displaying statistics, while the program itself might have functions for reading an array, printing an array, and the `main` function.

The steps are:

- The header file is **statistics.h**. It contains the prototypes. Since header files should be included only once per file, they should also contain guards (commands to the compiler)—see the example code.
- The library file is **statistics.c**. It just has the implementations.
- The main program must **#include** the header file. Your own header files are included with `"` rather than with `< >`. You should include the standard files before you include your header files.
- **gcc** must be told to compile the library file and join it to the main program. This is done by adding the name before the main program:

```
gcc statistics.c testStats.c -lm
```

23.3 Example Code

Here are the three files.

```
/* statistics.h */
#ifndef STATISTICS_H
#define STATISTICS_H 1
#endif
```

```
float mean(float[], int);
float sdev(float[], int);
```

```
/* statistics.c */
#include <math.h>
#include "statistics.h"
```

```
float mean(float data[], int size) {
    double sum=0.0;
    int i;
    for(i=0; i<size; i++)
        sum += data[i];
    return sum / size;
}
```

```
float sdev(float data[], int size) {
```

```
double sumDevs=0.0;
float mu = mean( data, size);
int i;
for(i=0; i<size; i++)
    sumDevs += (data[i] - mu)*(data[i]-mu);
return sqrt( sumDevs/ (size-1) );
}
```

```
/* testStats.c */
#include <stdio.h>
#include "statistics.h"

int main(void) {
    float array[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    int arraySize = 10;
    printf("Mean is %.2f\n", mean(array, arraySize));
    printf("Standard deviation is %.2f\n", sdev(array, arraySize));
    return 0;
}
```

BitOps

24.1 Bitwise Operators

A fundamental C approach is that one uses `ints` to store things other than numbers. An integer has a representation in **binary** as a series of **bits**: 1s and 0s. Sometimes a program uses an integer where each bit is a piece of information about the system, such as an error code: the first bit might say what type.

To access these bits you have bit operators. The most common are the ***shift*** operators. Shift right is equivalent to divide by 2, and shift left is equivalent to multiply by 2 (at least for small positive numbers). For example, the following code rounds a number down to the nearest even number:

```
x = x >> 1;  
x = x << 1;
```